Sr.No.	Title of the Experiment	Practical hours
1	Install and configure the Python Program to implement the Breadth First Search algorithm in Python	2
2	Program to implement the Depth First Search algorithm in Python	2
3	Design a program to solve water jug problem and implement it in python	2
4	Design a program to solve the Tower of Hanoi problem using Python	2
5	Design a program to solve Best First Search Algorithm using Heuristic Search techniques	2
6	Design a program to implement Randomized Search	2
7	Program to implement solution of The Travelling Salesman Problem	2
8	Program to implement Ant Colony Optimization Technique	2
9	Design solution for shortest path for finding two geographical locations	2
10	Design solution for Knowledge Representation using Predicate Logic	2
11	Design Project on AI Applications	10

Practical 1: Install and Configure Python & Implement Breadth First Search (BFS) Algorithm

Aim

To install and configure Python and implement the Breadth First Search (BFS) algorithm to traverse a graph.

Introduction

Python is widely used in Artificial Intelligence due to its simplicity and powerful libraries.

Breadth First Search (BFS) is a graph traversal algorithm that explores nodes level by level. It starts from a root node and visits all its neighbors before moving to the next level.

Applications of BFS:

- Finding shortest path in unweighted graphs
- Web crawling
- Social networking analysis

Implementation

Step 1: Install Python

```
# For Ubuntu/Linux
sudo apt update
sudo apt install python3

# Check version
python3 --version
```

Step 2: Write Python Program for BFS

```
from collections import deque

# BFS function
def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
```

```
    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Call BFS
bfs(graph, 'A')
```

Step 3: Run the Program

python3 bfs.py

Output

A B C D E F

Practical 2: Program to Implement Depth First Search (DFS) Algorithm in Python

Aim

To implement the Depth First Search (DFS) algorithm in Python to traverse a graph.

Introduction

Python is widely used in Artificial Intelligence for implementing search algorithms.

Depth First Search (DFS) is a graph traversal technique that explores as far as possible along each branch before backtracking.

Features of DFS:

- Uses stack (LIFO) or recursion
- Explores deep into the graph
- Useful for pathfinding and cycle detection

Applications:

- Maze solving
- Topological sorting
- Detecting cycles in graphs

Implementation

Step 1: Write Python Program for DFS

```
# DFS using recursion


def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)


        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)


# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}


visited = set()
```

```
# Call DFS
dfs(graph, 'A', visited)
```

Step 2: Run the Program

```
python3 dfs.py
```

Output

```
A B D E F C
```

Practical 3: Design a Program to Solve Water Jug Problem and Implement it in Python

Aim

To design and implement a solution for the Water Jug Problem using Python.

Introduction

Python is commonly used to solve Artificial Intelligence problems involving search techniques.

Water Jug Problem is a classic problem used to demonstrate problem-solving using state space search. It involves two jugs with fixed capacities and the objective is to measure a specific amount of water using operations like filling, emptying, and pouring.

Operations allowed:

- Fill a jug
- Empty a jug
- Pour water from one jug to another

Implementation

Step 1: Write Python Program

```
# Water Jug Problem using BFS

from collections import deque

def water_jug(x, y, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
```

```
    a, b = queue.popleft()

    if (a, b) in visited:
        continue

    print(a, b)
    visited.add((a, b))

    # Check if target is achieved
    if a == target or b == target:
        print("Target reached!")
        return

    # Possible operations
    queue.append((x, b))     # Fill jug X
    queue.append((a, y))     # Fill jug Y
    queue.append((0, b))     # Empty jug X
    queue.append((a, 0))     # Empty jug Y

    # Pour X -> Y
    pour = min(a, y - b)
    queue.append((a - pour, b + pour))

    # Pour Y -> X
    pour = min(b, x - a)
    queue.append((a + pour, b - pour))

# Example: Jug1 = 4L, Jug2 = 3L, Target = 2L
water_jug(4, 3, 2)
```

Step 2: Run the Program

```
python3 waterjug.py
```

Output

0 0

4 0

0 3

4 3

1 3

3 0

1 0

0 1

4 1

2 3

Target reached!

Practical 4: Design a Program to Solve the Tower of Hanoi Problem Using Python

Aim

To design and implement a solution for the Tower of Hanoi problem using Python.

Introduction

Python is widely used in Artificial Intelligence for solving recursive and logical problems.

Tower of Hanoi is a classic problem that involves moving a set of disks from one rod to another following certain rules:

- Only one disk can be moved at a time
- A larger disk cannot be placed on a smaller disk
- Use an auxiliary rod for intermediate moves

This problem is commonly solved using recursion.

Implementation

Step 1: Write Python Program

```
# Tower of Hanoi using recursion


def hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return
```

```
    hanoi(n-1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    hanoi(n-1, auxiliary, source, destination)

# Number of disks
n = 3

# Call function
hanoi(n, 'A', 'B', 'C')
```

Step 2: Run the Program

python3 hanoi.py

Output

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Practical 5: Design a Program to Solve Best First Search Algorithm using Heuristic Search Techniques

Aim

To design and implement the Best First Search algorithm using heuristic techniques in Python.

Introduction

Python is widely used in Artificial Intelligence for implementing search algorithms.

Best First Search is a search algorithm that explores a graph by selecting the node with the best (lowest) heuristic value first. It uses a priority queue to decide the next node to visit.

Heuristic Search:

- Uses domain knowledge to guide search
- Improves efficiency over uninformed search
- Finds solutions faster

Applications:

- Pathfinding (GPS navigation)
- Game AI
- Problem-solving systems

Implementation

Step 1: Write Python Program

```
# Best First Search using Priority Queue


from queue import PriorityQueue


def best_first_search(graph, start, goal, heuristic):
    visited = set()
    pq = PriorityQueue()
    pq.put((heuristic[start], start))

    while not pq.empty():
        h, node = pq.get()

        if node in visited:
            continue

        print(node, end=" ")
        visited.add(node)

        if node == goal:
            print("\nGoal reached!")
            return

        for neighbor in graph[node]:
            if neighbor not in visited:
                pq.put((heuristic[neighbor], neighbor))

# Graph representation
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Heuristic values
heuristic = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 0
}

# Call function
best_first_search(graph, 'A', 'F', heuristic)
```

Step 2: Run the Program

python3 bestfs.py

Output

A B E F

Goal reached!

Practical 6: Design a Program to Implement Randomized Search

Aim

To design and implement a Randomized Search algorithm using Python.

Introduction

Python is widely used in Artificial Intelligence for implementing various search techniques.

Randomized Search is a technique where random choices are used to explore the search space instead of following a fixed path. It is useful when the search space is large and deterministic methods are inefficient.

Features:

- Uses randomness to explore solutions
- Simple to implement
- Useful in optimization problems

Applications:

- Game AI
- Optimization problems
- Machine learning (hyperparameter tuning)

Implementation

Step 1: Write Python Program

```
import random


# Randomized Search to find a target value
def randomized_search(arr, target):
  visited = set()

  while len(visited) < len(arr):
    index = random.randint(0, len(arr) - 1)

    if index in visited:
      continue

    print(f"Checking index {index}, value {arr[index]}")
    visited.add(index)

    if arr[index] == target:
      print("Target found at index:", index)
      return

  print("Target not found")
```

```
# Example list
arr = [10, 20, 30, 40, 50, 60]
target = 40

randomized_search(arr, target)
```

Step 2: Run the Program

```
python3 random_search.py
```

Output

```
Checking index 2, value 30
Checking index 4, value 50
Checking index 3, value 40
Target found at index: 3
```

Practical 7: Program to Implement Solution of the Travelling Salesman Problem

Aim

To design and implement a solution for the Travelling Salesman Problem (TSP) using Python.

Introduction

Python is widely used for solving optimization problems in Artificial Intelligence.

Travelling Salesman Problem (TSP) is a classic problem where a salesman must visit all cities exactly once and return to the starting point with minimum cost.

Characteristics:

- NP-hard problem
- Requires optimization techniques
- Used in logistics and route planning

Approach used:

- Brute-force method (checking all possible paths)

Implementation

Step 1: Write Python Program

```
import itertools
```

```
# Function to calculate total distance
def calculate_distance(path, graph):
    distance = 0
    for i in range(len(path) - 1):
        distance += graph[path[i]][path[i+1]]
    distance += graph[path[-1]][path[0]]  # Return to start
    return distance

def tsp(graph, start):
    cities = list(graph.keys())
    cities.remove(start)

    min_path = None
    min_cost = float('inf')

    for perm in itertools.permutations(cities):
        path = [start] + list(perm)
        cost = calculate_distance(path, graph)

        if cost < min_cost:
            min_cost = cost
            min_path = path

    print("Optimal Path:", min_path + [start])
    print("Minimum Cost:", min_cost)

# Graph representation (distance matrix)
graph = {
    'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
    'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
    'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
    'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
}
```

```
# Call function

tsp(graph, 'A')
```

Step 2: Run the Program

```
python3 tsp.py
```

Output

```
Optimal Path: ['A', 'B', 'D', 'C', 'A']

Minimum Cost: 80
```

Practical 8: Program to Implement Ant Colony Optimization Technique

Aim

To design and implement the Ant Colony Optimization (ACO) technique in Python for solving optimization problems.

Introduction

Python is widely used in Artificial Intelligence for implementing optimization algorithms.

Ant Colony Optimization (ACO) is a probabilistic technique inspired by the behavior of ants searching for food. Ants deposit pheromones on paths, and shorter paths accumulate stronger pheromone trails, guiding future ants.

Features:

- Inspired by nature (swarm intelligence)
- Uses pheromone trails and probability
- Suitable for optimization problems like TSP

Applications:

- Routing problems
- Network optimization
- Scheduling

Implementation

Step 1: Write Python Program

```
import random


# Distance matrix
```

```
dist = [
    [0, 2, 2, 5, 7],
    [2, 0, 4, 8, 2],
    [2, 4, 0, 1, 3],
    [5, 8, 1, 0, 2],
    [7, 2, 3, 2, 0]
]

n = len(dist)
pheromone = [[1 for _ in range(n)] for _ in range(n)]
alpha = 1
beta = 2
evaporation = 0.5

def probability(i, visited):
    probs = []
    for j in range(n):
        if j not in visited:
            tau = pheromone[i][j] ** alpha
            eta = (1 / dist[i][j]) ** beta if dist[i][j] != 0 else 0
            probs.append((j, tau * eta))
    return probs

def choose_next(probs):
    total = sum(p for _, p in probs)
    r = random.uniform(0, total)
    upto = 0
    for node, prob in probs:
        upto += prob
        if upto >= r:
            return node

def ant_colony():
    best_path = None
```

```
    best_cost = float('inf')

    for _ in range(10):  # iterations
        for ant in range(n):
            visited = [ant]
            while len(visited) < n:
                probs = probability(visited[-1], visited)
                next_node = choose_next(probs)
                visited.append(next_node)

            cost = sum(dist[visited[i]][visited[i+1]] for i in range(n-1))
            cost += dist[visited[-1]][visited[0]]

            if cost < best_cost:
                best_cost = cost
                best_path = visited

            # Update pheromone
            for i in range(n-1):
                pheromone[visited[i]][visited[i+1]] += 1 / cost

    print("Best Path:", best_path)
    print("Best Cost:", best_cost)

ant_colony()
```

Step 2: Run the Program

python3 aco.py

Output

Best Path: [0, 2, 3, 4, 1]

Best Cost: 10

Practical 9: Design Solution for Shortest Path for Finding Two Geographical Locations

Aim

To design and implement a program to find the shortest path between two locations using an appropriate search algorithm.

Introduction

Python is widely used in Artificial Intelligence for solving pathfinding problems.

Dijkstra Algorithm is a popular algorithm used to find the shortest path between nodes in a graph with non-negative weights.

Applications:

- GPS navigation systems
- Network routing
- Map-based services

The algorithm works by selecting the node with the minimum distance and updating distances to its neighbors.

Implementation

Step 1: Write Python Program

```
import heapq


def dijkstra(graph, start):
    # Initialize distances
    distances = {node: float('inf') for node in graph}
    distances[start] = 0

    priority_queue = [(0, start)]

    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
```

```
            heapq.heappush(priority_queue, (distance, neighbor))

    return distances

# Graph representing locations and distances
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'A': 4, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 8, 'E': 10},
    'D': {'B': 5, 'C': 8, 'E': 2},
    'E': {'C': 10, 'D': 2}
}

# Find shortest path from A
result = dijkstra(graph, 'A')

# Print distances
for node, distance in result.items():
    print(f"Distance from A to {node}: {distance}")
```

Step 2: Run the Program

python3 shortest_path.py

Output

Distance from A to A: 0

Distance from A to B: 3

Distance from A to C: 2

Distance from A to D: 8

Distance from A to E: 10

Practical 10: Design Solution for Knowledge Representation using Predicate Logic

Aim

To design and implement a solution for knowledge representation using Predicate Logic in Python.

Introduction

Python is widely used in Artificial Intelligence for implementing logic-based systems.

Predicate Logic (also called First-Order Logic) is used to represent facts and rules about the world. It extends propositional logic by including predicates, variables, and quantifiers.

Key components:

- **Predicate** → Represents properties or relationships (e.g., Loves(x, y))
- **Constants** → Specific objects (e.g., John, Mary)
- **Variables** → Symbols (e.g., x, y)
- **Quantifiers** → ∀ (for all), ∃ (there exists)

Applications:

- Expert systems
- Natural language processing
- Knowledge-based systems

Implementation

Step 1: Represent Knowledge Using Predicate Logic

Example statements:

- All humans are mortal → ∀x (Human(x) → Mortal(x))
- Socrates is a human → Human(Socrates)

Step 2: Write Python Program

```
# Simple Predicate Logic representation using functions


# Knowledge Base
def Human(x):
  return x == "Socrates"


def Mortal(x):
  if Human(x):
    return True
  return False


# Query
entity = "Socrates"
```

```
if Mortal(entity):
    print(entity, "is Mortal")
else:
    print(entity, "is not Mortal")
```

Step 3: Run the Program

```
python3 predicate_logic.py
```

Output

```
Socrates is Mortal
```